RICE UNIVERSITY

Optional UI: a pipeline dashboard

Show, in a browser, why each article was kept, skipped, filtered or deduplicated.

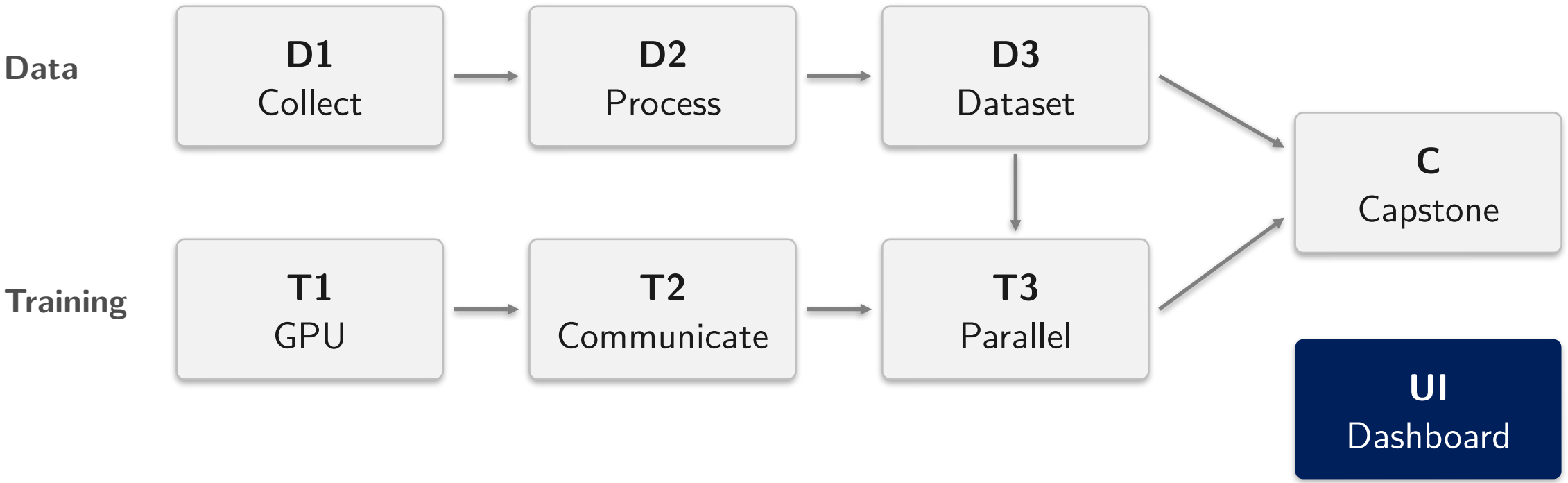
COMP 329 course staff

Rice University

Spring 2027 · Optional UI

Optional extension · No grade weight · 4–6 hours · JavaScript + Node

Where this assignment fits



In our pipeline

This assignment: `sample_runs/` → **UI** → `starter/app.js`

What you will build, and why

Make a saved pipeline run explainable to a person, safely and without stale data.

1. **Real API**

   Load status and lineage from the supplied HTTP server.

2. **Safe rendering**

   Corpus text is data, never HTML that runs.

3. **Async state**

   A slow old response must never overwrite a newer one.

What goes in, what comes out



Files you produce

`app.js`
Your three functions

`index.html, style.css`
Supplied; edit only on purpose

`Screenshots`
Loading, error and real-data states

Three functions, three milestones

`requestJSON(path, opts)`

`fetch → JSON → check response.ok`

Throw the API's message or the HTTP status.

`renderLineage(el, rows)`

`Clear, then build one article per row`

Use textContent, never innerHTML.

`loadRun(id, elements)`

`Show loading, fetch, render`

Drop responses from older selections.

Key idea

A title like <script>hostile()</script> must appear as literal text and never run.

Stale responses: the fast-switch bug

1

Select run A

request #1 starts (slow).

2

Select run B

request #2 starts (fast).

3

B returns

Render B. Latest is #2.

4

A returns late

#1 ≠ latest → ignore it.

5

Errors too

A late failure must not replace B's state.

Key idea

A simple incrementing request number is enough to tell whether a newer selection happened.

What you write, what you get

You write

`requestJSON`, `renderLineage`, `loadRun`

`starter/app.js` · numbered comments in order

Provided

- HTTP server (course.py run serve)
- index.html and style.css
- Event listeners
- Node-based tests

Ground rules

Keep the signatures · don't edit `starter/pipeline_lab` · run through `course.py` or `lab.py`.

Build it in this order

1. **requestJSON**
   Success and error paths.
2. **renderLineage**
   Literal text, sensible empty titles.
3. **loadRun**
   Loading state, stale-response guard.
4. **Real browser**
   Throttle, switch fast, keyboard, narrow screen.

```
check ui
```

```
run serve
```

Check

Done when: Both tests pass and the dashboard works on real saved data.

If something goes wrong

Browser never sees edits
Check: Edit `08_ui_dashboard/starter/app.js`; disable cache.

Old run appears after switching
Check: Check the request number before any DOM update.

A title becomes HTML
Check: Use textContent, not innerHTML.

Port already in use
Check: Stop the old viewer or pick another port.

Python relative-import error
Check: Launch through course.py.

Tests pass, page broken
Check: Node tests don't replace a real browser check.

The performance study

Does it work for a real person?

Predict first

List the states a user can hit: loading, error, empty, slow switch.

Real data
select a complete run, compare with the API

Stress
throttle the network and switch quickly

How to measure: Tab through with the keyboard, resize to a narrow screen, and record screenshots and console errors.

How you are graded

25	25	25	25
API + errors	Literal lineage	Async state	Browser evidence

Proposed extension rubric · no additional syllabus grade weight

Check

Done when: Both tests pass and the dashboard works on real saved data.

Your first ten minutes

```
$ bash setup.sh
$ source .venv/bin/activate
$ python course.py preflight
$ python course.py check ui
$ python course.py run serve \
    --runs 08_ui_dashboard/sample_runs \
    --port 8001
# then open http://localhost:8001
```

Your folder

`08_ui_dashboard/`

Handout:
`08_ui_dashboard/README.md`